Supplementary material: From Uncertainty Estimation to Repair Allocation

Melih Elibol, Ayşenur Gönül and Sushant Kumar Pandey

Abstract

This supplement records supporting aggregates, measurement controls, and accounting details for the main paper. All values are derived from the persisted result records in the project repository. Numerical variants from different feature configurations are kept separate rather than reconciled.

S1. Scope and cohort definitions

The BigCodeBench extension (Section S3) is a separate uncertainty experiment and is not repair evidence.

The corrected-representation ablation is used for the main AUC table. Earlier predictor, selection, and selective-risk arms remain explicitly distinct. In particular, sample-level text–execution AUC on CG/MBPP is 0.86 in one feature variant and 0.88 in the corrected family ablation; the two values are not interchangeable.

The MBPP cells use a fixed 60-problem cohort: the first 60 MBPP problems that are also present in MBPP+, selected so that EvalPlus probe inputs exist for every problem. In the uncertainty, selection and allocation tables, correctness labels for this cohort are the base MBPP test suite. Where strengthened labels are reported, they are named explicitly; MBPP+ is a re-authored test suite rather than a superset of the base assertions, so its counts do not decompose as the base counts minus verifier false positives. The held-out repair experiment uses a separate MBPP split and is listed separately in Table 2 of the main text.

S2. Measurement controls and calibration

Table S1 records representative representation corrections and residual associations. These are Pearson correlations, whereas the BigCodeBench ladder below reports Spearman correlations. They must not be combined into one effect-size comparison.

Table S1: Correlation with pass rate before and after prompt removal. Partial values control for prompt and completion length.

Cell	Metric	Original	Corrected	Partial
CG/HE164	Token Jaccard	−0.219	−0.548	−0.404
	Token cosine	−0.259	−0.341	−0.189
	AST sequence	−0.580	−0.581	−0.434
DS/HE60	Token Jaccard	−0.508	−0.769	−0.583
	Token cosine	−0.588	−0.632	−0.445
	AST sequence	−0.740	−0.723	−0.505

Mean completion-length correlation is −0.412 on CG/HE164 and −0.708 on DS/HE60. Length recovers 71–98% of AST-sequence correlation magnitude across seven cells. This comparison is with AST correlation, not with the best prediction AUC or all uncertainty information.

Table S2: Calibration examples. ECE values depend on task, binning, feature arm, and evaluation population.

Task	Cell / configuration	Rows	Bins	Raw ECE	Beta ECE
Problem	HE60 / reduced logistic	60	4	0.042	0.080
Problem	HE164 / no-CodeBERT	164	5	0.180	0.053
Sample	HE60 / text-core logistic	600	8	0.199	0.039
Sample	HE164 / text-core logistic	1640	11	0.226	0.016

Beta fitting is nested within the outer grouped folds, using inner out-of-fold predictions (main text, Section 3.2). A split-half attenuation calculation yields a reliability-based range of roughly 0.80–0.90 at ten samples. Because metric and pass-rate estimates share samples, classical independent-measurement-error assumptions do not strictly hold, so this range is an approximation rather than a ceiling.

S3. BigCodeBench extension

The environment gate accepts 997 of 1,140 reference solutions in the Kaggle image and 936 in the laptop environment. Forty mutated controls fail in each gate. This does not certify exhaustive test adequacy. The actual sampled 60-problem sets from the two environments overlap on only six tasks; a fixed task-set hash is therefore essential to comparisons across model runs. All five reported rungs use one recorded Kaggle subset.

Table S3: BigCodeBench: 60 environment-eligible problems, ten samples per problem, temperature 0.7, seed 42, CUDA/float16. Stars mark BH-FDR significance at 0.05 for correlations. Metric values use the full-code representation (prompt included); Table 3 in the main text uses the corrected representation, so the two must not be compared cell by cell.

	CG-350M	Qwen-0.5B	Qwen-1.5B	Qwen-3B	Qwen-7B
Mean pass fraction	.0450	.1183	.2800	.3433	.4067
Problems with a pass	11	20	38	36	41
Parse-failure fraction	.0700	.0150	.0067	.0017	.0000
AST-sequence ρ	−.190	−.266*	−.438*	−.162	−.153
Token-Jaccard ρ	−.207	−.272*	−.334*	−.316*	−.309*
Token-cosine ρ	−.229	−.379*	−.426*	−.349*	−.302*
Lexical-diversity ρ	.061	−.317*	−.270*	−.103	−.106

The extension supports distribution-sensitive metric rankings. It does not identify a causal effect of model size: the ladder shares one family above CodeGen, one subset, one seed, and one sample budget. No tree-edit-distance values are available. Eligibility depends on installed libraries and runtime conditions, and excluded tasks may systematically differ from included tasks. The numbers are not full BigCodeBench leaderboard results.

Table S4: Held-out MBPP repair accounting. The stop-on-pass column is a replay of the fixed-budget candidate order.

Quantity	Fixed three candidates	Stop-on-pass replay
Initial failures	50	50
Repaired failures	12	12
Model calls	150	132
Calls per added pass	12.5	11.0
Unique candidates (fixed arm)	136	–
Duplicate candidates (fixed arm)	14	–

S4. Repair and prompt comparisons

Eight successes arise on candidate zero, two on candidate one, and two on candidate two. The candidate-order replay explains the 18-call reduction; no alternative random draws were generated. The 50-case cohort is distinct from the 54-failure joint MBPP pool.

Table S5: HumanEval context ablation: paired comparisons. L-only and R-only denote discordant successes. The strengthened-label analysis contains 147 rather than 148 coordinates.

	Base label			Strengthened label		
Pair	L-only	R-only	Adjusted p	L-only	R-only	Adjusted p
None vs code	44	18	.0013	41	21	.011
None vs error	49	12	.0000036	47	14	.000052
Code vs error	16	5	.017	16	3	.0039

Strengthened-label success counts are 82, 62, and 49 for none, code, and error. `HumanEval/32` has no usable EvalPlus entry and is excluded from both label sets, so the strengthened-label rates run over 147 coordinates. The MBPP base-label counts are 27, 14, and 13 of 54. The code-only versus code-plus-error discordances are 3–2 under base labels and 2–3 under strengthened labels, so neither a consistent direction nor equivalence is established by that comparison.

S5. Allocation: coverage, calls, and attempts

Table S6: Coverage-matched random replays. Mean ± standard deviation and observed ranges are from 200 gates over a fixed trace pool.

Cohort	Policy	Added passes	Calls
MBPP	Repair-All	10	147
	Smart@75	7	120
	Random@75	7.56 ± 1.19 [4,10]	110.1 ± 3.5 [102,121]
HumanEval	Repair-All	54	368
	Smart@75	36	312
	Random@75	40.5 ± 2.7 [33,47]	276.1 ± 5.9 [261,290]

Smart and random select the same number of initial coordinates, not the same number of failures. All initial passing coordinates are skipped without a repair call. As a result, a gate that selects more failures can incur higher cost at the same nominal coverage.

Table S7: HumanEval code-plus-error pool: gains / attempted failures at common call caps. Random rows are rounded means over 200 replays. Reversal and shortest-first are exploratory policies.

Policy	25%	40%	50%	75%
Ascending pass score	7/34	9/53	14/67	30/106
Random	13.5/37	21.3/59	26.8/74	40.1/110
Reversed pass score	24/42	36/66	40/80	47/114
Shortest first	27/45	37/67	40/80	51/115

Caps are fractions of 368 repair calls. Figure 4 of the main paper connects these four aggregates only. A trace replay knows the eventual cost of each stored attempt, whereas an online system cannot know it before running the attempt; the curves are therefore retrospective replays, not online budget guarantees.

S6. Reproduction

The replication package (https://github.com/Melotachi/llm-reliability-replication; archived at https://doi.org/10.5281/zenodo.23698173) includes `make_figures.py` and `figure_data.json`, which regenerate the two vector plots from the reported aggregates. The Wilson intervals in the context figure are recalculated from counts; no confidence bands, random-replay observations, or candidate outcomes are synthesized.

The repository additionally contains the problem identifiers, source-run joins, test versions, model checkpoints, split assignments and random-policy replay code needed to audit every analysis; its `README.md` gives the environment, the exact reproduction command for every analysis, and a map from each table and figure to its script and result file.